

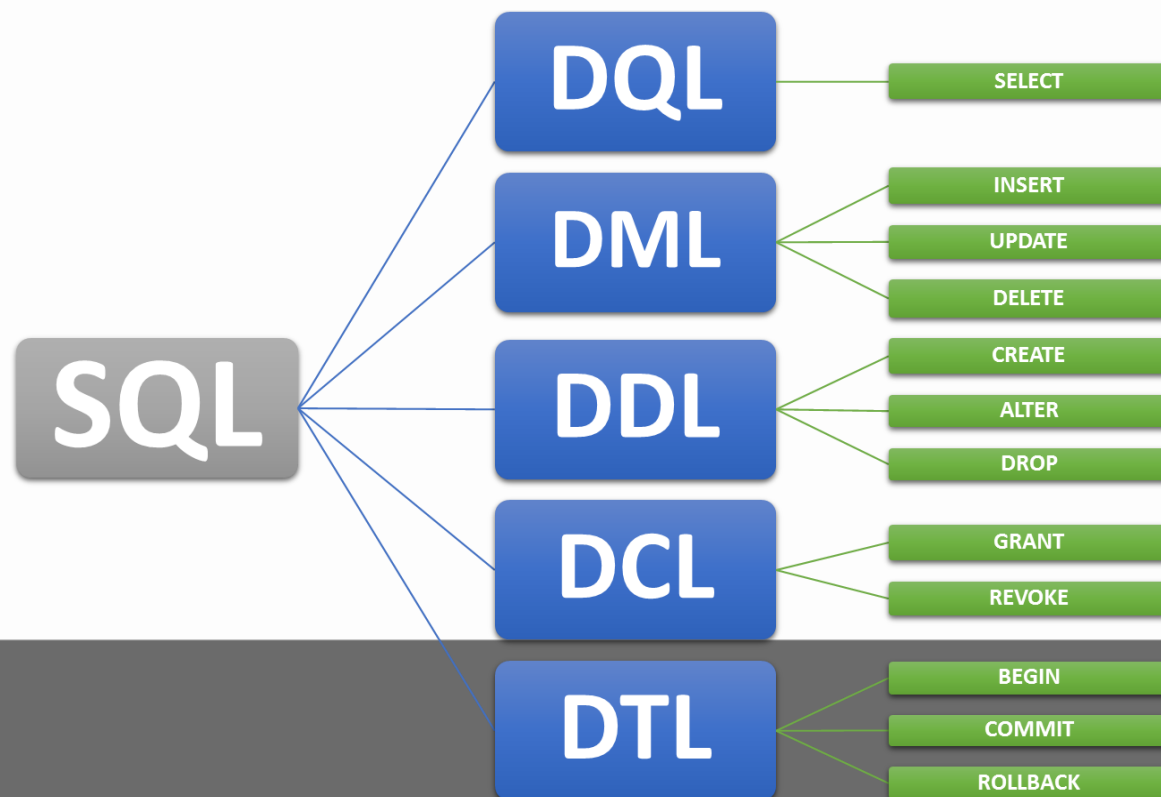
# Banco de dados

## Linguagem SQL

### DQL – Data Query Language

- ☐ Introdução
- ☐ Instrução Select
  - ☐ Cláusula Where
  - ☐ Cláusula Like
  - ☐ Operadores comparativos
  - ☐ Cláusula Order By
  - ☐ Cláusula Between
  - ☐ Alias

A linguagem SQL é o recurso mais conhecido por DBAs e programadores para a execução de comandos em bancos de dados relacionais. É por meio dela que criamos tabelas, colunas, índices, atribuímos permissões a usuários, bem como realizamos consultas a dados. Enfim, é utilizando a SQL que “conversamos” com o banco de dados.



O comando select é bastante simples e, serve para você, mostrar os dados da tabela. Sua sintaxe pode ser utilizado de algumas formas, como:

**SELECT \* FROM** NOME\_DA\_TABELA;

Exemplo: **SELECT \* FROM** alunos;

**Nota 1:** o asterisco indica que você deseja mostrar todos os campos, ou seja, id, nome, email e cidade (conforme a tabela do MySQL que você deseja exibir os registros).

Exemplo: **SELECT** cpf, nome, telefone **FROM** alunos;

**Nota 2:** neste caso, você está indicando que deseja mostrar somente nome e e-mail. Isso é interessante, pois a consulta é executada de forma mais rápida, já que você está trazendo menos informações da tabela.

Os operadores de comparação são usados em condições que comparam uma expressão a outro valor ou expressão. A tabela abaixo mostra os operadores:

- = Igual a
- > Maior que
- >= Maior ou igual a que
- < Menor que
- <= Menor ou igual a que
- <> Diferente de

```
SELECT codigo_empregado, nome, salario  
      FROM empregados  
      WHERE codigo_empregado = 8
```

## Cláusula Where e seus Operadores no SQL



O uso de um operador lógico faz com que duas condições tenham de produzir um resultado único. Uma linha só poderá ser retornada se o resultado global da condição for verdadeiro.

- **AND** Retorna TRUE se ambas as condições forem verdadeiras
- **OR** Retorna TRUE se uma das condições for verdadeira
- **NOT** Retorna TRUE se a condição seguinte for falsa

**SELECT** nome, salário, cidade, estado

**FROM** empregados

**WHERE** estado = 'SP' **AND** salario > 2200

| OPERADOR | EXEMPLO                   |
|----------|---------------------------|
| AND      | CONDIÇÃO 1 AND CONDIÇÃO 2 |
| OR       | CONDIÇÃO 1 OR CONDIÇÃO 2  |
| NOT      | NOT CONDIÇÃO              |

A cláusula Where permite ao comando SQL passar condições de filtragem.

## Exemplos:

-- Selecione o registro do aluno cujo cpf seja igual a 11111111111

❑ **SELECT** \* from alunos **WHERE** cpf = 11111111111;

-- Selecciona o registro do aluno cuja altura seja igual a 1.75

❑ **SELECT** \* from alunos **WHERE** altura = 1.75;

-- Selecciona o registro dos alunos com cpf maior que 22222222222

❑ **SELECT** \* from alunos **WHERE** cpf > 22222222222;

-- Selecciona o registro cuja data seja maior que 01/01/2020

❑ **SELECT** \* from alunos **WHERE** datanasc > '2020-01-01'

**WHERE** com **AND** e **OR** – Usamos os operadores **AND** e **OR** junto com Where quando é necessário usar mais de uma condição de comparação.

- ☐ O operador **AND** é usado quando desejamos atender as duas condições distintas.

**SELECT \* FROM** alunos

**WHERE** nome = 'Fulano de Tal' **AND** altura > 1.75

- ☐ O operador **OR** é usado especificamente quando desejamos atender uma ou outra condição.

**SELECT \* FROM** alunos

**WHERE** nome = 'Fulano de Tal' **OR** nome = 'Beltrano'



Quando realizamos uma consulta no MySQL, utilizamos a cláusula **WHERE** para realizar um filtro dos registros a retornar.

Porém, com o **WHERE**, só podemos aplicar filtros de correspondência exata de palavras. E se precisarmos aplicar um filtro que verifique palavras de forma parcial, como palavras que iniciem ou terminem com determinados caracteres, ou que possuam sequências de caracteres específicas? Neste caso, usamos a cláusula **LIKE**:

```
SELECT * FROM alunos  
      WHERE nome LIKE '%A' AND altura > 1.75
```

A cláusula **LIKE** determina se uma cadeia de caracteres (string) corresponde a um padrão especificado. Um padrão pode incluir caracteres normais e curingas.

❑ **SELECT \* FROM** alunos

**WHERE** nome **LIKE** 'Wagner%' **AND** altura > 1.70

❑ **SELECT \* FROM** alunos

**WHERE** nome **LIKE** '%A%' **AND** altura > 1.70



MySQL Like & Not  
Like Wildcard  
Operator With  
Example

A cláusula **NOT LIKE** inverte a comparação, verificando se a cadeia de caracteres NÃO corresponde ao padrão especificado.

☐ **SELECT \* FROM** livro  
**WHERE** sobrenome **NOT LIKE** 'F%';

☐ **SELECT \* FROM** livro  
**WHERE** Nome\_Livro **NOT LIKE** 'S%';

Podemos usar o operador **IN** no lugar do operador **OR** em determinadas situações. O **IN** permite verificar se o valor de uma coluna está presente em uma lista de elementos.

-- **WHERE** com **OR**

```
SELECT * FROM clientes  
    WHERE cpf = 33333333333  
    OR cpf = 44444444444
```

-- **WHERE** com **IN**

```
SELECT * FROM clientes  
    WHERE cpf IN (33333333333, 44444444444)
```

Este operador, ao contrário do **IN**, permite obter como resultado o valor de uma coluna que não pertence a uma determinada lista de elementos.

-- **WHERE** com **AND**

```
❑ SELECT * FROM clientes  
    WHERE cpf = 333333333333  
    AND cpf = 444444444444
```

-- **WHERE** com **NOT IN**

```
❑ SELECT * FROM clientes  
    WHERE cpf NOT IN (333333333333, 444444444444)
```

O operador BETWEEN tem a finalidade de permitir a consulta entre uma determinada faixa de valores.

-- **WHERE com AND**

```
❑ SELECT * FROM clientes  
    WHERE cpf = 33333333333  
    AND cpf = 44444444444
```

-- **WHERE com BETWEEN**

```
❑ SELECT * FROM clientes  
    WHERE cpf BETWEEN 11111111111 AND 99999999999
```

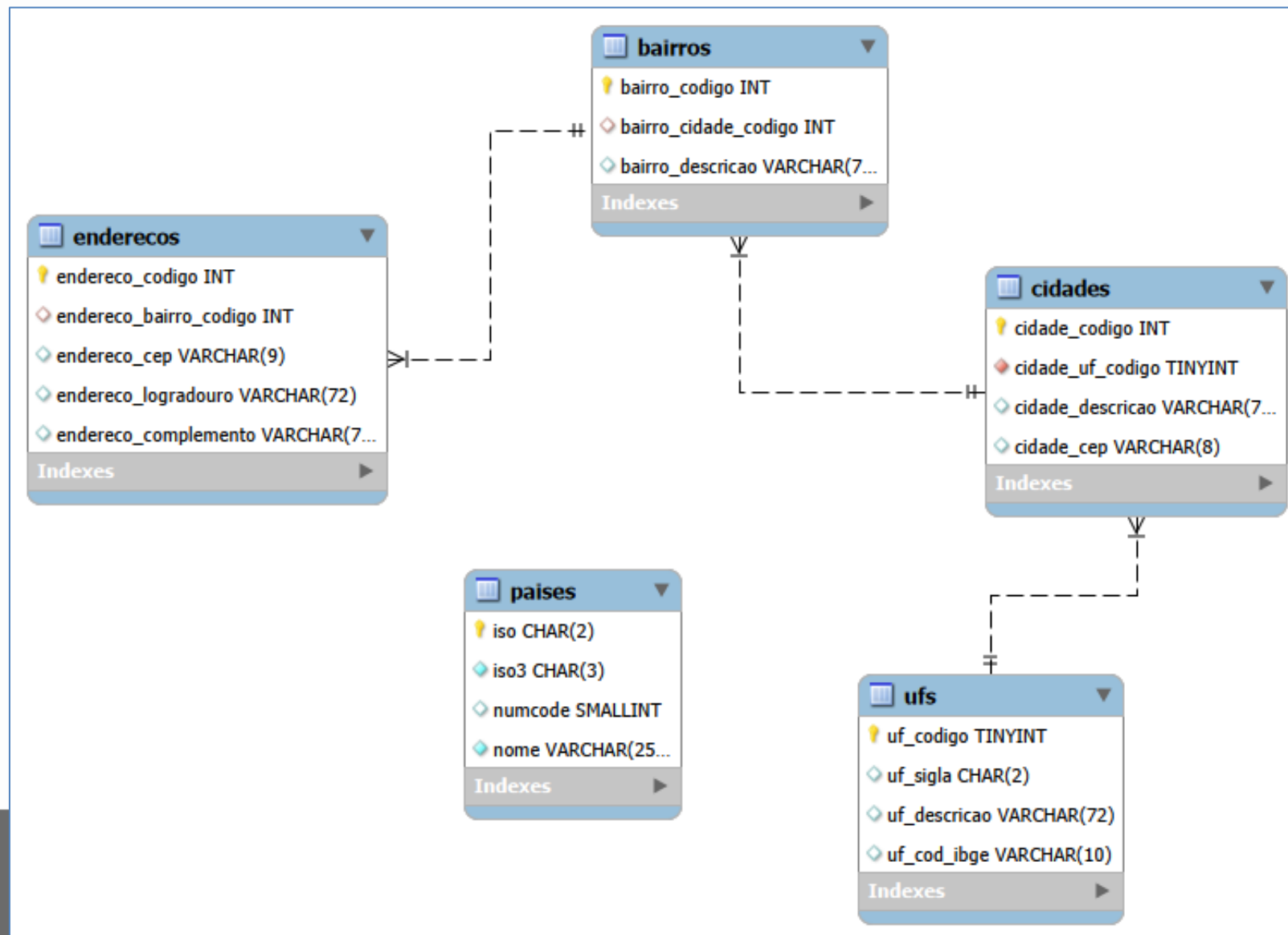
O operador NOT BETWEEN, ao contrário do anteriormente descrito, permite consultar os valores que não se encontram em uma determinada faixa.

-- **WHERE** com **AND**

```
❑ SELECT * FROM clientes  
    WHERE cpf = 33333333333  
    AND cpf = 44444444444
```

-- **WHERE** com **BETWEEN**

```
❑ SELECT * FROM clientes  
    WHERE cpf NOT BETWEEN 11111111111 AND 99999999999
```





1. Consulte e retorne todos os registros da tabela cep\_cidade;
2. Selecione todos os registros das cidades que começam com a letra B e informe a quantidade encontrada;
3. Selecione todas as cidades do estado do PR;
4. Selecione todas as cidades cujo código do IBGE esteja entre 22222222 e 33333333;
5. Consulte e retorne todos os bairros cujo id da cidade seja igual a 125;

6. Consulte e retorne todos os bairros cujo id da cidade seja menor que 100;
7. Consulte e retorne todos os bairros da cidade de Bebedouro - SP da tabela cep\_endereco;
8. Consulte e retorne todos os ceps da cidade de Pitangueiras - SP da tabela cep\_faixa\_cidades;
9. Selecione a latitude e longitude das cidades de Bebedouro, Pitangueiras, Monte Azul Paulista e Viradouro;
10. Selecione todas as faixas de cep do estado do GO e que esteja entre os valores 74491000 e 74492000.

